

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	Lella Thannveer Reddy	Reg. No.:	192524293
Date:	04/08/2026	Duration:	60 minutes

Code of Conduct

I _Lella Thannveer Reddy/192524293_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _Lella Thannveer Reddy_

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms. □ Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

1. Analyze System Requirements

System Objectives

- Develop a software system that is reliable, secure, and user-friendly.
- Automate business processes and reduce manual work.
- Improve data accuracy, efficiency, and decision-making.
- Provide fast access to information for authorized users.

Functional Requirements

- User registration, login, and authentication.
- Add, edit, delete, and search records.
- Generate reports and dashboards.
- Store and retrieve data from the database.
- Send notifications and maintain activity logs.

Non-Functional Requirements

- High performance with quick response time.
- Secure access through authentication and authorization.
- Easy maintenance and software updates.
- High availability with minimal downtime.
- Scalability to support future growth.

Key Quality Attributes

Scalability – Supports increasing users and data.

Maintainability – Modular design allows easy updates.

Reliability – Provides consistent and accurate results.

Availability – System remains accessible whenever required.

Performance – Processes requests efficiently.

Security – Protects sensitive data using encryption, authentication, and access control.

2. Select a Suitable Software Architecture

Selected Architecture: Layered Architecture

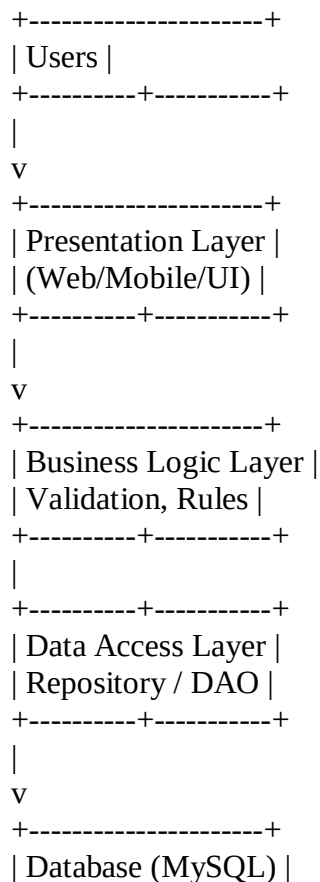
Architecture Layers

1. Presentation Layer – User interface.
2. Business Logic Layer – Implements application rules.
3. Data Access Layer – Handles database communication.
4. Database Layer – Stores application data.

Justification

- Clear separation of responsibilities.
- Easy testing and debugging.
- High code reusability.
- Simple integration with APIs.
- Suitable for web, desktop, and mobile applications.
- Supports long-term maintenance and future enhancements.

3. Software Architecture Diagram



+-----+

External APIs / Email / Payment / Cloud Services

<----- REST APIs ----->

4. Explain Components and Their Interactions

Presentation Layer

- Displays the user interface.
- Collects user input and sends requests.

Business Logic Layer

- Validates input.
- Applies business rules.
- Coordinates system operations.

Data Access Layer

- Performs CRUD operations.
- Converts application requests into database queries.

Database

- Stores user accounts, transactions, reports, and logs.

Component Interaction

1. User submits a request.
2. Presentation layer forwards the request.
3. Business logic validates and processes it.
4. Data access layer communicates with the database.
5. Database returns results.
6. Business logic prepares the response.
7. Presentation layer displays the output.

Communication Mechanism

- HTTP/HTTPS between client and server.
- REST APIs for service communication.
- SQL queries for database access.

5. Discuss Quality Attributes

Scalability

- New users, modules, and servers can be added without redesigning the system.

Security

- User authentication and authorization.
- Password encryption and secure communication (HTTPS).
- Role-based access control.

Performance

- Efficient database queries.
- Caching and optimized business logic reduce response time.

Reliability

- Exception handling, logging, and backup improve system stability.
- Data validation prevents incorrect information.

Maintainability

- Modular architecture simplifies debugging and upgrades.
- Independent layers reduce the impact of code changes.

Availability

- Database backup and server redundancy improve uptime.
- Recovery mechanisms minimize service interruption.

6. Justify Architectural Decisions

Rationale

Layered Architecture was selected because it separates the user interface, business logic, and database operations into independent layers. This improves readability, modularity, testing, and maintenance.

Trade-offs

Advantages

- Easy maintenance.
- Better security.
- Reusable components.
- Easier testing and debugging.
- Supports team development.

Disadvantages

- Additional communication between layers may introduce slight performance overhead.
- More project structure compared to a simple monolithic design.

Support for Future Enhancements

- New features can be added without changing the entire system.
- Easy integration with cloud services, REST APIs, payment gateways, and third-party applications.
- Supports migration to microservices if the application grows.

Long-Term Maintainability

- Independent modules simplify updates.
- Reduced code duplication.
- Better documentation and easier onboarding for developers.
- Supports continuous integration and deployment practices.

Conclusion

The proposed Layered Architecture provides a balanced solution by ensuring scalability, security, performance, reliability, maintainability, and availability. It is suitable for most software applications and can easily adapt to future business and technical requirements.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design	Produces a clear, complete, and wellstructured architecture diagram with all major components,	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient	Diagram is incomplete, unclear, or technically incorrect.
	interfaces, and interactions accurately represented. Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability,		detail. Provides limited explanation of interactions and addresses only some quality attributes.	
Component Interaction and Quality Attribute Analysis		Explains most components and quality attributes with minor gaps.		Component interactions and quality attributes are poorly explained or

				missing.
	maintainability, and availability.			
Architectural Justification and Technical Presentation	Provides strong justification for architectural Provides adequate justification with minor gaps; report is well organized.	with minimal decisions, discusses discussion of trade-offs and future trade-offs or enhancements, and future scope; presents a well-	Limited justification report lacks technical clarity.	Justification is weak or absent; organization and
	organized,		report needs	
	professional report.		improvement.	

Criteria	Mark
System Requirement Analysis	/5
Selection of Software Architecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and Quality Attribute Analysis	/5
Architectural Justification and Technical Presentation	/5
TOTAL	/5